A3-PS013-TC203

Incident Timelines and Evidence Collection for Small Teams

Veena G (mentor)
Veenag@vemanait.edu.in
Huda Akhtar (team member 1)
hudabir786@gmail.com
M Yogitha Chowdary (team member 2)
yogithachowdary05@gmail.com

AIORI Hackathon 3

October 14, 2026

Problem Description

Cybersecurity logs are scattered across systems, formats, time zones, and unaligned clocks, making incident reconstruction difficult. Manual investigation is time-consuming and error-prone, while preserving original evidence is essential for trustworthiness.

- Relevant events are spread across different sources, such as Linux authentication logs and web-server access logs.
- Manual searching and comparing timestamps is slow, repetitive and vulnerable to human error.
- Different time zones and inaccurate system clocks can make event order difficult to establish.
- Original log files must remain unchanged so investigators can review the source evidence.
- Unparseable lines can hide useful clues if they are silently discarded.

Project goal: build a simple, affordable workflow that normalises events, preserves originals originals and produces a searchable timeline and report.

Background

Why incident timeline reconstruction matters

1. **Logs are fragmented**: Each system records its own view of activity.
2. **Time is inconsistent:** Time zones and clock drift can distort event order.
3. **Evidence needs care:** Original files and integrity checks support review.

Scope of our prototype

•Input: two log formats — Linux authentication/system logs and web-server access logs.
•Processing: extract timestamps, source, IP address and event details; preserve original timestamps while normalising to UTC.
•Output: chronological timeline, search/filter interface and an incident report with file hashes and parsing gaps.
•Evaluation: three simulated scenarios; compare investigation time against a manual baseline.

Literature & RFC Review

1. **NIST SP 800-61 Rev. 2:** Computer Security Incident Handling Guide-Provides an incident-response lifecycle and highlights preparation, analysis, documentation and lessons learned.
2. **RFC 3339:** Date and Time on the Internet: Timestamps-Defines a standard timestamp format with UTC and numeric offsets; supports consistent time representation.
3. **MITRE ATT&CK:** Knowledge base of adversary tactics and techniques-Useful vocabulary for describing simulated behaviours such as brute-force attempts and privilege escalation.
4. **Python hashlib / SQLite documentation:** Hashing and lightweight structured storage-Supports SHA-256 digest generation and a local database for normalised event records.

Existing guidance explains how incident response and evidence handling should be performed, but small teams still need a lightweight tool that combines log ingestion + preservation + hashing + timestamp normalization + timeline search + reporting

Proposed Solution

Lightweight Incident Timeline & Evidence Collection Tool

Our system accepts raw logs from **two different log formats** and processes them through a common forensic workflow.

1. Log Collection: Accept raw host/application log files and preserve the original records.

2. Evidence Integrity: Generate SHA-256 hash and associate it with collected evidence. •

3. Log Parsing: Extract timestamp, source, IP address and event/message. •

4. Timestamp Normalization: Convert timestamps to UTC, retain the original timestamp, and account for timezone differences and clock skew. •

5. Timeline Reconstruction: Combine events chronologically and store normalized events in SQLite. •

6. Investigation & Reporting: Search/filter by time, source, IP or keyword; identify parsing gaps; export an incident summary report.

Optimization Proposed

Manual Investigation vs Proposed Tool

Area	Manual / before	Proposed / after
Event correlation	Open files separately and compare by eye	One searchable chronological timeline
Time handling	Mixed formats / local times can confuse ordering	UTC-normalised timestamps; original time retained
Evidence integrity	Original-file handling may be inconsistent	Originals retained with SHA-256 hashes
Missing data	Unclear or unreadable lines may be skipped	Parsing gaps are explicitly recorded
Reporting	Notes compiled manually	Report bundles timeline, hashes and gaps

Solution Architecture & Design

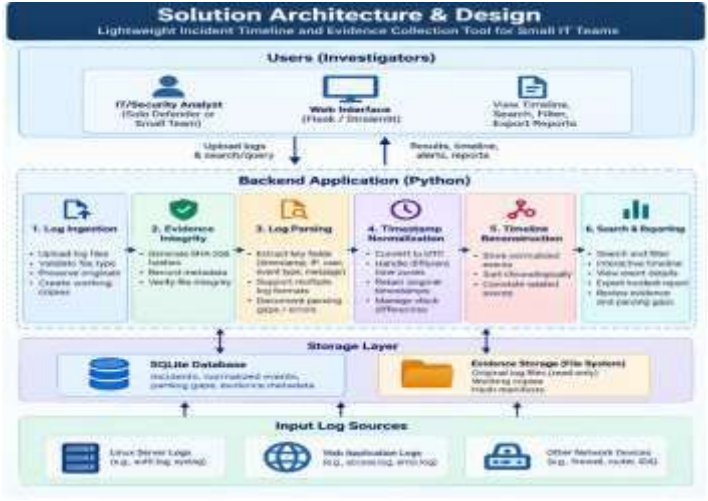


Figure: System Architecture UML Diagram

Data Flow Diagram

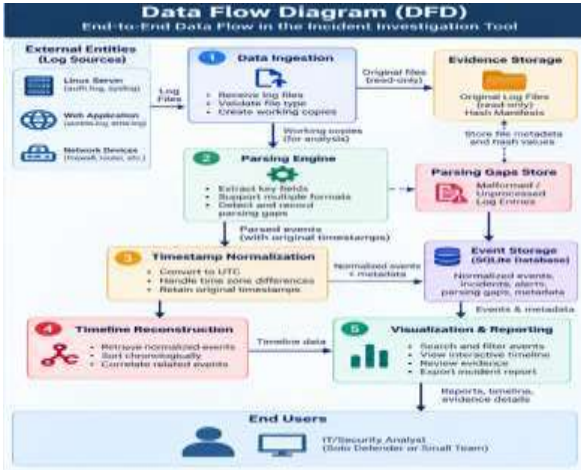


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Clearly define the deployment milestones and methodology.

Week 1: Planning & environment setup
Week 2: Log ingestion, parsing & SHA-256 hashing
Week 3: Timeline reconstruction & dashboard
Week 4: Testing, validation & deployment

Methodology: Modular development, iterative testing, and local deployment using Python, Flask/Streamlit, and SQLite.

Bibliography I

1. NIST, Computer Security Incident Handling Guide, SP 800-61 Rev. 2, 2012. https://doi.org/10.6028/NIST.SP.800-61r2

2. G. Klyne et al., Date and Time on the Internet: Timestamps, RFC 3339, 2002. https://www.rfc-editor.org/rfc/rfc3339

3. MITRE, ATT&CK Knowledge Base. https://attack.mitre.org/

4. Python Software Foundation, hashlib — Secure hashes and message digests. https://docs.python.org/3/library/hashlib.html

5. SQLite, SQLite Documentation. https://www.sqlite.org/docs.html

6. Python Software Foundation, datetime — Basic date and time types. https://docs.python.org/3/library/datetime.html

Thank You for Your Attention!